

Comparador de AFD

Autómatas Finitos Deterministas · Equivalencia · Minimización · Simulación

Integrantes: Wendy Yolanny Jaimes Lugo
Jonathan David Quintero Villamizar

Asignatura: Automas y Lenguajes Formales

Docente: Jhon Sebastián Gomez Sierra

Introducción

- Un AFD es un modelo formal de cómputo.
- Definido por la 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$.
- Reconoce exactamente los lenguajes regulares.
- Base teórica de compiladores, regex y validadores.
- Útil para enseñar la noción de cómputo determinista.

Problema planteado

- Comparar AFD a mano es propenso a errores.
- Verificar equivalencia requiere un procedimiento formal.
- Falta de herramientas visuales y didácticas.
- Necesidad de generar contraejemplos verificables.
- Soporte para aprender minimización paso a paso.

Objetivos

- General: construir una herramienta web para trabajar AFD.
- Editar y validar AFD en tiempo real.
- Simular cadenas paso a paso con animación.
- Minimizar AFD por refinamiento de particiones.
- Verificar equivalencia con BFS sobre el producto.
- Visualizar todo como grafos interactivos.

¿Qué es un AFD?

Definición formal

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : conjunto finito de estados

Σ : alfabeto finito

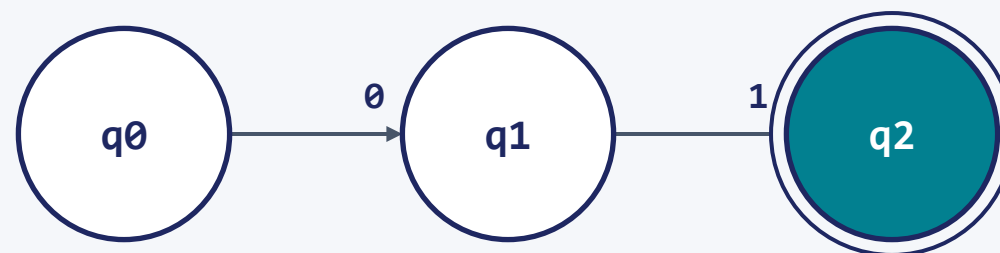
$\delta: Q \times \Sigma \rightarrow Q$ (función de transición)

$q_0 \in Q$: estado inicial

$F \subseteq Q$: estados finales o de aceptación



Ejemplo: termina en 01

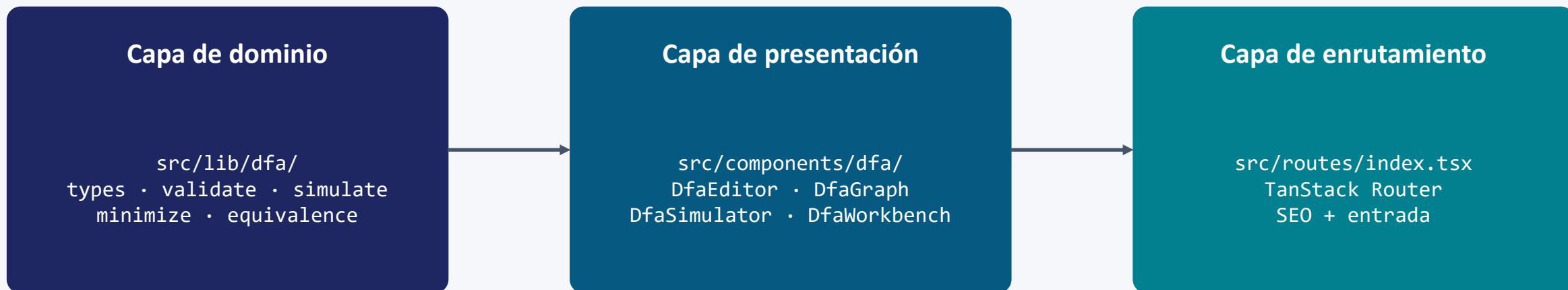


Estado inicial: q_0 Final: q_2 Alfabeto: $\{0,1\}$

Funcionamiento de la aplicación

- Header con tema claro/oscuro y resumen de estado.
- Pestaña Editor: tabla de transiciones + grafo en vivo.
- Pestaña Comparar: análisis de equivalencia.
- Pestaña Minimizar: original vs. mínimo.
- Pestaña Simular: ejecución animada paso a paso.
- Importar / exportar AFD en formato JSON.

Arquitectura del sistema



Stack tecnológico

React 19 + TypeScript · TanStack Start v1 (Vite 7)

Tailwind CSS v4 · shadcn/ui · Radix UI

React Flow (visualización de grafos) · Lucide Icons · Sonner toasts

Explicación del código · tipo central

Interfaz DFA — el contrato del sistema

```
interface DFA {  
    states: string[];  
    alphabet: string[];  
    transitions: Record<string, Record<string, string>>;  
    initial: string;  
    finals: string[];  
}
```

La función de transición δ se modela como un mapa anidado estado $\rightarrow$ símbolo $\rightarrow$ estado. Esta representación garantiza acceso $O(1)$, serialización JSON natural y validación trivial del determinismo.

Funciones principales



validateDFA(dfa)

Detecta errores estructurales tipados.



simulate(dfa, input)

Ejecuta δ paso a paso sobre una cadena.



minimize(dfa)

Refinamiento de particiones (Moore).



checkEquivalence(a, b)

BFS sobre el autómata producto.

Algoritmo · Simulación

- Paso 1: current = q_0 ; lista de pasos vacía.
- Paso 2: para cada símbolo a de la cadena:
 - – si $a \notin \Sigma \rightarrow$ error.
 - – si $\delta(\text{current}, a)$ no existe $\rightarrow$ error.
 - – registrar paso y actualizar current.
- Paso 3: aceptar si current $\in F$ al final.

Algoritmo · Comparación de equivalencia

Idea: avanzar ambos AFD en paralelo (autómata producto)

1. Inicializar cola con (q_0^A, q_0^B) .
2. Extraer par (p, q) y comprobar pertenencia a F .
3. Si $(p \in F_1) \neq (q \in F_2) \rightarrow$ **NO equivalentes; reportar cadena.**
4. Para cada $a \in \Sigma$: encolar $(\delta_1(p,a), \delta_2(q,a))$ si no se visitó.
5. Si la cola se vacía sin diferencias $\rightarrow$ **EQUIVALENTES.**

Complejidad

$$O(|Q_1| \cdot |Q_2| \cdot |\Sigma|)$$

Cada par de estados se visita a lo sumo una vez. BFS garantiza encontrar el contraejemplo más corto.

Resultado en la app: muestra ✓ Equivalentes o ✗ No equivalentes + cadena contraejemplo.

Algoritmo · Minimización

- Paso 1: eliminar estados inalcanzables desde q_0 .
- Paso 2: partición inicial $P = \{ F, Q - F \}$.
- Paso 3: calcular firma de cada estado = vector de particiones destino.
- Paso 4: subdividir cada grupo según firmas distintas.
- Paso 5: repetir hasta el punto fijo (sin nuevas divisiones).
- Paso 6: cada partición es un estado Q_i del AFD mínimo.

Visualización gráfica

- Grafo dirigido renderizado con React Flow.
- Layout circular calculado dinámicamente.
- Nodos personalizados: inicial ($\rightarrow$), final (anillo).
- Aristas múltiples agrupadas con etiquetas (a, b).
- Resaltado en tiempo real durante la simulación.
- Mini-mapa, zoom y arrastrar nodos.

Demostración

Flujo de la demo

1. Cargar 'Termina en 01' en AFD 1.
2. Cargar 'Número par de ceros' en AFD 2.
3. Ver las tarjetas de validez verdes.
4. Ir a Simular: probar '1101' en AFD 1 $\Rightarrow$ ACEPTADA.
5. Ir a Comparar: la app declara NO equivalentes.
6. Observar el contraejemplo '01'.
7. Ir a Minimizar: el AFD par-ceros ya es mínimo.

Resultados obtenidos

- Aplicación 100% funcional, lista para producción.
- Validación en tiempo real con errores tipados.
- Simulación animada con historial de cadenas.
- Minimización correcta con eliminación de inalcanzables.
- Equivalencia formal con contraejemplos verificables.
- Interfaz responsiva, accesible, con tema claro/oscuro.

Conclusiones

- Los algoritmos clásicos siguen siendo elegantes y eficaces.
- Los contraejemplos formales son potentes didácticamente.
- React Flow simplifica la visualización de grafos complejos.
- Mejoras futuras: $AFND \rightarrow AFD$, $regex \leftrightarrow AFD$, exportar LaTeX/TikZ.

*Gracias por su
atención*